

9 种高级 RAG 技术及其实现方法



致Great

2025-11-11

AI

向量数据库

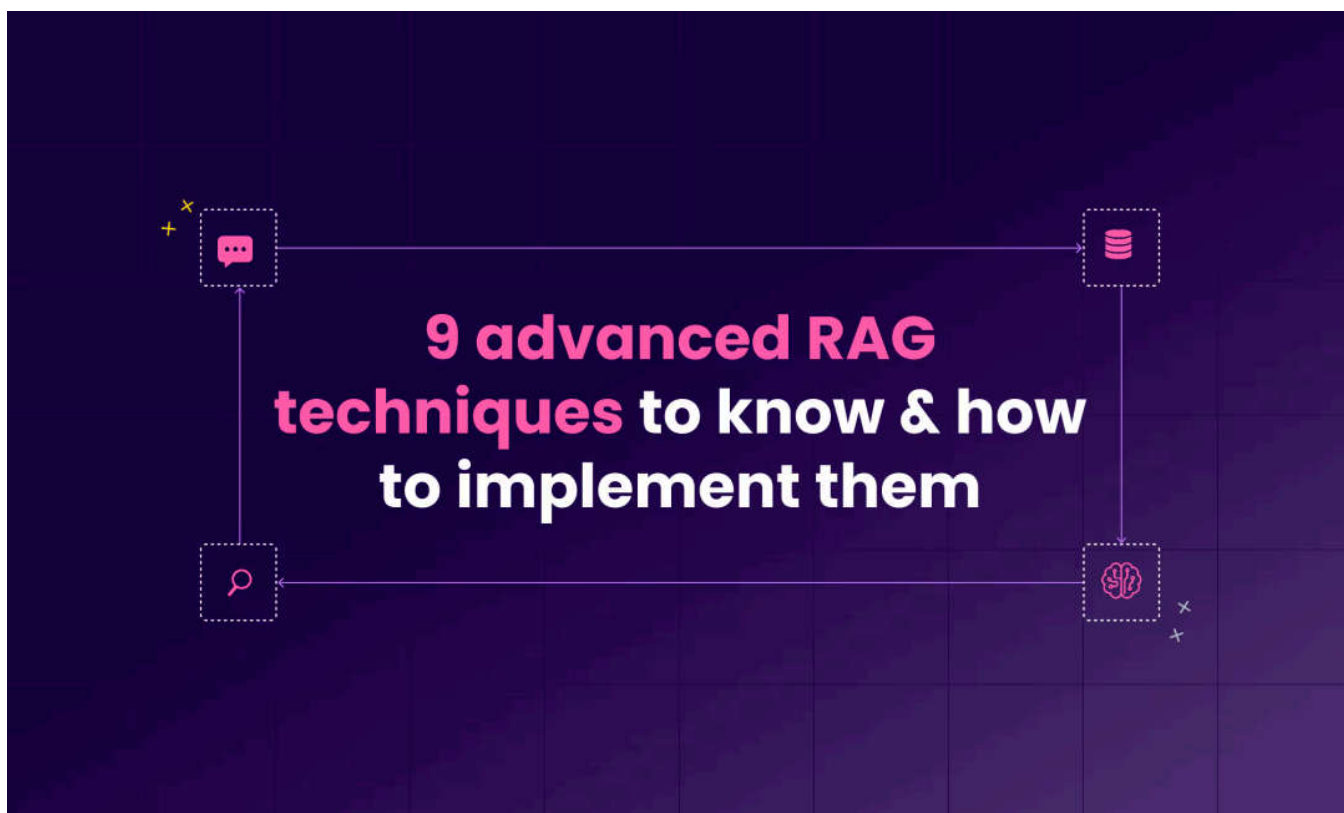
大模型

机器学习

推荐 机器学习平台

[了解详情 >](#)

机器学习平台是面向机器学习应用开发者，提供 WebIDE 和自定义训练等丰富建模工具、多框架高性...



本文将探讨 9 种关键的高级检索增强生成 (RAG) 技术，并介绍如何借助相关工具实现它们。

在实际的 AI 应用中，RAG 技术能有效优化 RAG 管道的准确性和灵活性。从更智能的文本分块（chunking）到混合搜索（hybrid search）和上下文蒸馏（context distillation），这些方法对于提供更快、更准确的响应至关重要。

我们为什么需要这些技术？因为基本的 RAG 系统常常面临结果嘈杂、上下文不相关或排序不佳等问题。当检索系统无法找到相关信息时，用户会对其失去信任，大型语言模型（LLM）也会浪费时间处理错误的上下文。

本文将详细介绍 9 种重要的先进 RAG 技术：

- 文本分块
- 重新排序

- 利用元数据
- 混合搜索
- 查询重写
- 自动裁剪 *上下文蒸馏
- 微调大型语言模型（LLM）
- 微调嵌入模型

你还将学习如何使用 Meilisearch、LangChain 和向量数据库等工具来实现这些策略，以及如何评估哪些策略能真正带来提升。

下面我们开始深入探讨。

Advanced RAG techniques

Technique	How it works
Text chunking	Breaks long documents into smaller pieces (“chunks”) that are easier to index and retrieve.
Reranking	Reordering results so the most relevant chunks rise to the top before being passed to the LLM.
Leveraging metadata	Using structured tags, like date, author, domain, or document type, to filter and boost the right results.
Hybrid search	Combines keyword-based (sparse) and vector-based (dense) retrieval methods to improve result quality.
Query rewriting	Reformulates vague or underspecified queries so the system can better understand intent and return more relevant results.
Autocut	Trims text dynamically based on token limits or model constraints, without cutting into semantic meaning.
Context distillation	Condenses large documents into high-value, meaningful summaries.
Fine-tuning the LLM	Involves retraining a pre-trained language model on your specific data to understand better the type of responses you want.
Fine-tuning the embedding models	Reshaping how the system “understands” similarity.



1. 文本分块

文本分块是将长文档切分成更小的片段（即“块”），以便于索引和检索。

RAG 管道处理的是这些文本块，而非整个文档，因此文本块的质量直接影响检索准确性和答案清晰度。

如果文本块过长，AI 模型可能会被干扰；如果过短，则可能丢失上下文。找到合适的平衡点有助于获得最佳结果。

以下是几种分块类型：

- **简单分块：** 将文档分割成固定大小且略有重叠的块（例如，35 个字符，重叠 5 个字符）。这种方法易于实现，但不考虑语义结构，可能将关键思想分割到不同块中。
- **语义分块：** 利用嵌入和余弦阈值，根据相似性对句子进行分组。这种方法能保持相关思想的完整性，提高检索精度，但需要像 BERT 这样的嵌入模型以及更高的计算性能。
- **基于语言模型的分块：** 使用大型语言模型（LLM）将文本分割成独立的语义单元。这种方法能产生高度连贯的结果，但计算成本高昂，可能需要进行微调。

不出所料，每种方法都有其权衡。如果处理的是产品手册或技术文档，语义分块或基于 LLM 的分块可能更具优势。对于更简单的内容，固定大小的分块就足够了。

2. 重新排序

在 RAG 系统中，初始检索器会获取大量的检索信息，其中有好的，也有不理想的。重新排序（Reranking）是指在将结果传递给大型语言模型（LLM）之前，对检索到的结果进行重新排列，使最相关的信息排在前面。

以下是几种常见的重新排序方法：

- **交叉编码器重新排序：** 将用户查询和每个候选文本块输入到一个 Transformer 模型（如 BERT）中，该模型会评估它们之间的匹配程度。这种方法非常准确，但速度慢且资源密集。当质量比延迟更重要时，此方法最适用。
- **基于分数重新排序：** 使用启发式方法或相关性分数（如 BM25 提升、关键词搜索匹配）调整排序。这种方法速度快、开销小，但精细度较低。

这两种技术都有效，但选择哪种取决于你能承受的延迟。

3. 利用元数据

利用元数据（Leveraging metadata）是指使用结构化标签，如日期、作者、领域或文档类型，来过滤和提升相关结果。

元数据过滤是一种投入低、效果显著的技术，因为它不需要大量的计算。

以下是其主要子类型：

***日期过滤：** 移除过时数据，只关注最新文档。这在金融等快速变化的领域非常有用。然而，过度关注最新内容有时可能会排除重要的历史文档。

- **作者/来源过滤：** 提升来自可信个人或来源的检索文档。这种方法产生幻觉（hallucination）的可能性很低，但你也可能因此错过一些不那么知名但高度相关的数据。
- **文档类型过滤：** 根据格式（例如，指南、博客或政策文件）进行过滤。虽然这有助于匹配用户意图，但如果元数据不一致，也存在过度过滤的风险。

4. 混合搜索

混合搜索（Hybrid search）结合了基于关键词（稀疏）和基于向量（密集）的检索方法，以提高结果质量。

这能显著提高相关性，尤其是在边缘情况或用户查询模糊时。

混合搜索主要有两种方法：

***分数融合：** 使用加权分数结合稀疏和密集方法的搜索结果。这种方法能实现平衡的准确性，但需要仔细调整以避免结果嘈杂。

- **结果合并：** 从两种方法中检索出最佳结果，然后合并并重新排序。这种方法实现简单，但存在重复或排序逻辑不一致的风险。

5. 查询重写

查询重写（Query rewriting）是在检索过程之前对用户输入进行优化。它会重新组织模糊或不明确的查询，使检索系统能更好地理解用户意图并返回更相关的结果。这对于处理简短或包含拼写错误的查询尤其有帮助。

大多数现代生成式 AI 系统都在其背后使用各种算法来优化查询。以下是几种类型：

- **同义词扩展：** 这种技术会自动添加等效词来覆盖更多变体。虽然这可以提高召回率，但也可能引入一些不相关的匹配。
- **拼写校正：** 自动修复任何错别字或拼写错误。这提升了普通用户的成功率，但也可能“纠正”一些小众或有效行话。
- *** 意图澄清：** 这一过程使用规则或机器学习将宽泛的查询转换为更清晰的查询。它能提高相关性，但需要训练数据并增加了复杂性。

6. 自动裁剪 (Autocut)

自动裁剪（Autocut）是根据 token 限制或模型约束动态地修剪文本，而不切断语义含义。这是一种在不移除关键信息的前提下，删除不那么相关上下文的方法。这很重要，因为许多大型语言模型（LLM）都有严格的 token 限制。如果上下文窗口过载，AI 模型可能会产生幻觉、跳过关键信息或表现不佳。

以下是两种常见的自动裁剪策略：

- **分数修剪：** 根据重要性对内容进行排名，并裁剪得分最低的部分。虽然优先级设定很智能，但这需要有一个有效的评分机制。
- **基于规则的修剪：** 应用固定规则（例如，删除引言段落或脚注）。这种方法易于实现，但缺乏对细微内容的适应性。

👍 点赞

💬 评论

☆ 收藏

↗ 分享

对于复杂的 RAG 系统而言，每次都解析大量文档并非最佳选择。上下文蒸馏（Context distillation）有助于将大型文档浓缩成高价值、有意义的摘要。

如你所知，大型语言模型（LLM）只需要“正确”的信息，而非“所有”信息。更多的上下文不一定意味着更好的结果，甚至可能稀释最终输出。上下文蒸馏有助于模型在生成响应时保持专注和准确。

有两种方法可以实现这一点：

- **基于摘要的蒸馏：** 使用提取式或抽象式摘要来保留关键点。这种方法产生简洁的上下文，但可能会遗漏细节。
- **问题驱动的蒸馏：** 根据具体问题定制上下文。这种方法高度相关，但依赖于准确的查询理解。

8. 微调大型语言模型（LLM）

微调大型语言模型（LLM）是指在你的特定数据集上重新训练预训练的语言模型，以便更好地理解你期望生成的响应类型。

当仅凭提示工程无法很好地指导模型时，微调尤其有用，特别是在受监管行业或特定领域背景下。

- **监督式微调：** 使用标记的问答、代码或指令进行再训练。此方法提供精度和控制，但需要精选数据和资源。
- **基于人类反馈的强化学习：** 人类对输出进行评分，模型学习偏好被认可的响应。此方法与价值观或品牌语调保持一致，但资源密集。

9. 微调嵌入模型

嵌入模型将文档转换为向量，以实现文档检索。微调嵌入模型意味着重塑检索模型对相似性的“理解”方式，这在现成的嵌入模型在你的领域表现不佳时至关重要。

- **对比学习：** 训练嵌入模型，使语义相似的文本更接近，而不相似的文本更远离。这提高了检索准确性，但需要标记的正向和负向样本。

- **领域适应：** 继续在你的语料库上进行训练，使嵌入模型与你的领域保持一致。这种方法更容易设置，特别是对于未标记的数据，但存在过拟合的风险。

微调大型语言模型（LLM）和嵌入模型可以让你完全控制检索和响应生成，为高性能、领域感知的 RAG 模型奠定基础。## 什么是高级 RAG 技术？

高级 RAG 技术超越了检索增强生成的基本原理。它们在文本分块、检索和内容排序方面更注重细节和细微之处。然而，最终目标是一致的：生成更好、更相关的搜索结果。

无论是优化混合搜索、重写查询，还是调整元数据，这些方法都基于检索增强生成的核心原则，旨在提高准确性和效率。

为什么需要高级 RAG 技术？

基本 RAG 设置的问题在于，它们常常难以处理复杂的查询。此外，在处理多样化数据集或在关键场景下保持相关性时，它们也常常力不从心。

随着用户期望的增长以及大型语言模型（LLM）在生产级工作流程中变得越来越核心，仅仅依靠暴力检索已经不够了。我们需要智能的 AI 应用来理解细微差别，并优先考虑来自可信来源的最新数据。

高级技术有助于你从“检索并希望”转向“有目的地检索”。它们解决了幻觉、延迟和内容不匹配等关键痛点，同时提高了准确性和性能。

如何在 RAG 中实现高级技术？

你可以使用不同的工具、库和框架来实现高级检索技术。

以下是一些可用于实现高级检索增强生成技术的 RAG 工具：

- Meilisearch
- Weaviate

让我们进一步了解它们：

1. Meilisearch

Meilisearch 是一个开源工具，在设计时充分考虑了用户体验，非常适合实现高级 RAG 管道。它支持可定制的 ranking 规则，并能轻松处理大量文档集合。

你可以使用 Meilisearch 的混合搜索插件实现混合搜索（结合关键词和向量搜索的相关性）。这可以平衡语义搜索理解与传统精度。


```
// 示例代码片段，用于演示 Meilisearch 的使用场景
// 索引大型文档为结构化块
// client.index('my\_index').addDocuments([
//   { id: 1, title: 'Chunk 1 of document A', content: '...' },
//   { id: 2, title: 'Chunk 2 of document A', content: '...' },
// ]);
```

当处理大型文档时，你可以通过将其作为结构化块进行索引来提高搜索质量：

```
// 示例代码片段，用于演示 Meilisearch 的使用场景
// 查询带有元数据的文档
// client.index('my\_index').search('my query', {
//   filter: 'document\_type = "policy"',
//   // boost results from trusted authors
//   rankingRules: ['author\_score:desc', 'typo', 'words', 'proximity', 'attribute',
//   }]);
```



Meilisearch 的性能和易于集成使其成为构建智能 RAG 管道的首选解决方案，而无需过度复杂化。

2. Weaviate

Weaviate 是一个原生向量搜索引擎，专为高性能语义搜索而构建。它直接与 Transformer 和嵌入模型集成，非常适合 RAG 实现。

3. LangChain

LangChain 是一个 Python/JavaScript 框架，它将大型语言模型（LLM）、检索组件和工具串联起来。它使 RAG 工作流程模块化且可配置。

你可以在一个统一的流程中插入重新排序模型、编写自己的查询重写器并管理分块管道。

4. Pinecone

Pinecone 是一个托管的向量数据库，专为大规模相似性搜索而构建。它适用于需要闪电般快速、可扩展的向量检索、重新排序和时间过滤查询的项目。

结合 Meilisearch 或 LangChain，Pinecone 为你的 RAG 设置增加了可扩展性和生产就绪能力。

如何评估 RAG 系统中的高级技术？

要评估高级 RAG 技术，你需要测试每项技术对检索质量的影响。目标是实现高精度、丰富的上下文、低延迟和可靠的实际可用性。

为了评估有效性，请关注以下指标：

- **检索准确性：** 是否返回了最具上下文相关性的文本块？这会影响下游的一切，包括幻觉率和答案质量。
- **延迟：** 重新排序或混合搜索等技术是否会拖慢你的系统？评估端到端响应时间。*
- **精确度与召回率：** 你返回的相关文档是太少（或太多）了吗？你需要找到平衡覆盖率和专注度的最佳点。
- **用户满意度：** 终端用户多久能准确获得他们需要的东西？考虑进行 A/B 测试或人工评估。

当你在不牺牲质量的前提下优化速度时，Meilisearch 快速准确的混合引擎脱颖而出。

为什么更智能的 RAG在实践中很重要？

高级 RAG 技术最终目的是让信息检索更智能，而不仅仅是更快。它们帮助系统超越暴力搜索，转向上下文感知检索，从而在不同领域实现扩展。

通过分层应用混合搜索、重新排序和上下文蒸馏等策略，团队可以设计出在生产环境中提供更精确、更可靠和更高效结果的 RAG 管道。

👁 105 🍌 0 💬 0 ☆ 0

关于作者



致Great

关注

评论

未登录

看完啦，[登录](#) 分享一下感受吧~

暂无评论